

Git Befehle Übersicht:

Grundlegende Befehle:

git init:	Initialisiert ein neues Git-Repository in einem bestehenden oder neuen Verzeichnis.
git clone <url>:	Kloniert ein bestehendes Git-Repository von einem entfernten Server auf deinen lokalen Computer.
git add <datei>:	Fügt die angegebenen Dateien zur Staging-Umgebung hinzu, bereitet sie also für den nächsten Commit vor.
git commit -m "Commit-Nachricht":	Erstellt einen neuen Commit mit den gestaged Änderungen und einer beschreibenden Nachricht.
git status:	Zeigt den aktuellen Status des Repositorys an, einschließlich nicht-verfolgter Dateien, Änderungen, die zum Commit bereit sind, und den aktuellen Branch.
git log:	Zeigt den Commit-Verlauf an. Mit Optionen wie `--oneline` oder `--graph` kann die Ausgabe formatiert werden.

Arbeiten mit Branches:

git branch:	Verwaltet Branches.
git branch <name>:	Erstellt einen neuen Branch.
git checkout <name>:	Wechselt zu einem bestehenden Branch.
git branch -d <name>:	Löscht einen lokalen Branch.
git merge <branch>:	Führt die Änderungen aus dem angegebenen Branch in den aktuellen Branch zusammen.
git rebase <branch>:	Spielt die Commits eines Branches auf einen anderen auf.

Interaktion mit entfernten Repositories:

git remote add <name> <url>:	Fügt ein entferntes Repository hinzu.
git fetch <name>:	Holt Änderungen von einem entfernten Repository ab, ohne sie lokal zu integrieren.
git pull <name> <branch>:	Holt Änderungen von einem entfernten Branch ab und führt sie in den lokalen Branch zusammen.
git push <name> <branch>:	Überträgt lokale Änderungen an ein entferntes Repository.

Weitere nützliche Befehle:

git diff:	Zeigt die Unterschiede zwischen Commits, Arbeitsverzeichnis und Staging-Bereich an.
git reset <commit>:	Setzt den HEAD auf einen bestimmten Commit zurück.
git revert <commit>:	Hört eine Änderung rückgängig, indem ein neuer Commit erstellt wird, der die Änderungen des angegebenen Commits rückgängig macht.
git tag <name> <commit>:	Erstellt ein Tag für einen bestimmten Commit.

Tipps und Tricks:

.gitignore:	Eine Datei, die angibt, welche Dateien oder Verzeichnisse Git ignorieren soll.
Git-Alias:	Erstelle benutzerdefinierte Abkürzungen für häufig verwendete Befehle.
Git-GUI:	Nutze grafische Benutzeroberflächen wie GitKraken oder Sourcetree, um Git visuell zu bedienen.

Zusätzliche Informationen:

Git-Workflows:	Es gibt verschiedene etablierte Workflows wie Git Flow oder GitHub Flow, die dir helfen können, deinen Entwicklungsprozess zu strukturieren.
Git-Konfiguration:	Mit `git config` kannst du globale oder lokale Einstellungen anpassen.